

Report conversation

This metal code of kernal_mul_mat_q3_k doesn't work, but q5_k works. Can you compare the two, and find anything wrong with it? It is highly commented to give you some clue

```
#define QK_K 256

typedef struct {
    uint8_t hmask[QK_K/8]; // quants - high bit
    uint8_t qs[QK_K/4];    // quants - low 2 bits
    uint8_t scales[3*QK_K/64]; // scales, quantized with 6
    bits
    half d;                // super-block scale
} block_q3_k;
// 110 bytes / block
kernel void kernel_mul_mat_q3_k_f32(
    device const void * src0,
    device const float * src1,
    device float * dst,
    constant int64_t & ne00,
    constant int64_t & ne10,
    constant int64_t & ne0,
    constant int64_t & ne1,
    threadgroup float * sum [[threadgroup(0)]],
    uint2 tgpig[[threadgroup_position_in_grid]],
    uint2 tptg[[thread_position_in_threadgroup]],
    uint2 tptg[[threads_per_threadgroup]]) {

    const uint32_t kmask1 = 0x03030303;
    const uint32_t kmask2 = 0x0f0f0f0f;

    const uint8_t m3 = 3;
    const int8_t m4 = 4;

    const int nb = ne00/QK_K;

    const int64_t r0 = tgpig.x;
```

```

const int64_t r1 = tgpig.y;

device const block_q3_k * x = (device const block_q3_k
*) src0 + r0*nb;
device const float * yy = (device const float *) src1 +
r1*ne10;

const int nth = tptg.x*tptg.y;
const int ith = tptg.y*tpitg.x + tpitg.y;

const int tid = tpitg.y;
const int il = tid/4; // 0...3 0 -> 0...63, 1 ->
64...127, 2 -> 128...191, 3 -> 192...255
const int ip = il / 2; // 0 or 1 0 -> use 1st 32 q's
(0...127), 1 -> 2nd 32 (128...255)
const int is = il % 2; // 0 or 1 0 -> 0...63, 128...191,
1 -> 64...127, 192...255
const int ir = tid - 4*il; // 0...3
const int n = 4;
const int l0 = n * ir; // first index for this thread
within a group of 32 (0, 4, 8, 12)
// 0...31 use 1<<0, 32...63 use 1<<1, 64...95 use 1<<2,
96...128 use 1<<3, etc.
// we process 64*il...64*il+63 -> 1st mask is 1<<(2*il),
second is 1<<(2*il+1)
// masks for high bit
const uint8_t m = 1 << (2*il);
const uchar2 mask = {m, (uint8_t)(m << 1)};

const int shift1 = 4*ip; // 1st shift for scale. must be 0
(0...127) or 4 (128...255)
const int shift2 = 2*il; // 2nd shift for scale. 0, 2, 4, or 6
// 1st shift for quants must be 0 in 0...31, 2 in 32...64, 4
in 64...96, 6 in 96...128, then agsin 0, 2, etc.
const int shift3 = 4*is;
const int shift4 = shift3 + 2;

const int q_offset = 32*ip + l0;
const int y_offset = 64*il + l0;

```

```

float sumf = 0;
for (int i = tpitg.x; i < nb; i += tptg.x) {

    // Copied from the C de-quantization code
    //aux[0] = ((a[0] >> 0) & kmask2) | (((a[2] >> 0) &
kmask1) << 4);
    //aux[1] = ((a[1] >> 0) & kmask2) | (((a[2] >> 2) &
kmask1) << 4);
    //aux[2] = ((a[0] >> 4) & kmask2) | (((a[2] >> 4) &
kmask1) << 4);
    //aux[3] = ((a[1] >> 4) & kmask2) | (((a[2] >> 6) &
kmask1) << 4);

    /// 0....63 we need a[0] with shift=0, a[2] with shift 0
    /// 64...127 we need a[1] with shift=0, a[2] with shift
2
    ///128...191 we need a[0] with shift=4, a[2] with shift
4
    ///192...255 we need a[1] with shift=4, a[2] with shift
6
    /// a[is] >> (4*ip) & 0xF | a[2] >> (2*il) & 3
    device const uint32_t * a = (device const uint32_t
*)x[i].scales;
    const char4 sc = as_type<char4>(((a[is] >> shift1) &
kmask2) | (((a[2] >> shift2) & kmask1) << 4));

    // Here I was thinking "what if the above is not
processed correctly because x[i].scales is not 4-byte
// aligned?". If that was the issue, using a uint16_t
pointer should solve it as x[i].scales is 2-byte aligned.
    // It does not solve the problem, it just makes it run
slower.
    //device const uint16_t * a = (device const uint16_t
*)x[i].scales;
    //const char2 sc1 = as_type<char2>((uint16_t)
(((a[2*is+0] >> shift1) & kmask2) | (((a[4] >> shift2) &
kmask1) << 4)));
    //const char2 sc2 = as_type<char2>((uint16_t)
(((a[2*is+1] >> shift1) & kmask2) | (((a[5] >> shift2) &
kmask1) << 4)));

```

```

device const uint8_t * q = x[i].qs + q_offset;
device const uint8_t * h = x[i].hmask + l0;

device const float * y = yy + i * QK_K + y_offset;

const float dall = (float)x[i].d;

float4 sums = {0.f, 0.f, 0.f, 0.f};
for (int l = 0; l < n; ++l) {
    sums[0] += y[l+0] * ((int8_t)((q[l+0] >> shift3) & m3)
- ((h[l+0] & mask[0]) ? 0 : m4));
    sums[1] += y[l+16] * ((int8_t)((q[l+16] >> shift3) &
m3) - ((h[l+16] & mask[0]) ? 0 : m4));
    sums[2] += y[l+32] * ((int8_t)((q[l+0] >> shift4) &
m3) - ((h[l+0] & mask[1]) ? 0 : m4));
    sums[3] += y[l+48] * ((int8_t)((q[l+16] >> shift4) &
m3) - ((h[l+16] & mask[1]) ? 0 : m4));
}

sumf += dall * (sums[0] * (sc[0] - 32)
    + sums[1] * (sc[1] - 32)
    + sums[2] * (sc[2] - 32)
    + sums[3] * (sc[3] - 32));
//sumf += dall * (sums[0] * (sc1[0] - 32)
//    + sums[1] * (sc1[1] - 32)
//    + sums[2] * (sc2[0] - 32)
//    + sums[3] * (sc2[1] - 32));

}

sum[ith] = sumf;

//
// Accumulate the sum from all threads in the
threadgroup
//
threadgroup_barrier(mem_flags::mem_threadgroup);
if (ith%4 == 0) {
    for (int i = 1; i < 4; ++i) sum[ith] += sum[ith + i];
}

```

```

    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%16 == 0) {
        for (int i = 4; i < 16; i += 4) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith == 0) {
        for (int i = 16; i < nth; i += 16) sum[0] += sum[i];
        dst[r1*ne0 + r0] = sum[0];
    }
}

typedef struct {
    half d;          // super-block scale for quantized
scales
    half dmin;       // super-block scale for quantized
mins
    uint8_t scales[3*QK_K/64]; // scales and mins,
quantized with 6 bits
    uint8_t qh[QK_K/8];    // quants, high bit
    uint8_t qs[QK_K/2];    // quants, low 4 bits
} block_q5_k;
// 176 bytes / block
kernel void kernel_mul_mat_q5_k_f32(
    device const void * src0,
    device const float * src1,
    device    float * dst,
    constant int64_t & ne00,
    constant int64_t & ne10,
    constant int64_t & ne0,
    threadgroup float * sum [[threadgroup(0)]],
    uint2 tgpig[[threadgroup_position_in_grid]],
    uint2 tptg[[thread_position_in_threadgroup]],
    uint2 tptg[[threads_per_threadgroup]]) {

    const uint16_t kmask1 = 0x3f3f;
    const uint16_t kmask2 = 0x0f0f;
    const uint16_t kmask3 = 0xc0c0;

```

```
const int nb = ne00/QK_K;

const int64_t r0 = tgpig.x;
const int64_t r1 = tgpig.y;

device const block_q5_k * x = (device const block_q5_k
*) src0 + r0*nb;
device const float * yy = (device const float *) src1 +
r1*ne10;

const int nth = tptg.x*tptg.y;
const int ith = tptg.y*tpitg.x + tpitg.y;

const int tid = tpitg.y; // 0...16
const int il = tid/4; // 0...3
const int ir = tid - 4*il;// 0...3
const int n = 4;

const int im = il/2; // 0 or 1. 0 computes 0,32 + 128,160,
1 computes 64,96 + 192,224
const int in = il%2;

const int l0 = n*(2*ir + in);
const int q_offset = 32*im + l0;
const int y_offset = 64*im + l0;

const uint8_t hm1 = 1u << (2*im);
const uint8_t hm2 = hm1 << 1;
const uint8_t hm3 = hm1 << 4;
const uint8_t hm4 = hm2 << 4;

uchar2 sc1, sc2, sc3, sc4;

float sumf = 0;
for (int i = tpitg.x; i < nb; i += tptg.x) {

device const uint8_t * q1 = (x + i)->qs + q_offset;
device const uint8_t * q2 = q1 + 64;
device const uint8_t * qh = (x + i)->qh + l0;
device const float * y1 = yy + i*QK_K + y_offset;
```

```
device const float * y2 = y1 + 128;
```

```
const float dall = (float)((x + i)->d);
```

```
const float dmin = (float)((x + i)->dmin);
```

```
device const uint16_t * a = (device const uint16_t *) (x + i)->scales;
```

```
sc1 = as_type<uchar2>((uint16_t)(a[im+0] & kmask1));
```

```
sc2 = as_type<uchar2>((uint16_t)(a[im+2] & kmask1));
```

```
sc3 = as_type<uchar2>((uint16_t)((a[im+4] >> 0) & kmask2) | ((a[im+0] & kmask3) >> 2));
```

```
sc4 = as_type<uchar2>((uint16_t)((a[im+4] >> 4) & kmask2) | ((a[im+2] & kmask3) >> 2));
```

```
float4 s = {0.f, 0.f, 0.f, 0.f};
```

```
float smin = 0;
```

```
for (int l = 0; l < n; ++l) {
```

```
    s[0] += y1[l+0] * ((q1[l] & 0xF) + (qh[l] & hm1 ? 16 : 0));
```

```
    s[1] += y1[l+32] * ((q1[l] >> 4) + (qh[l] & hm2 ? 16 : 0));
```

```
    s[2] += y2[l+0] * ((q2[l] & 0xF) + (qh[l] & hm3 ? 16 : 0));
```

```
    s[3] += y2[l+32] * ((q2[l] >> 4) + (qh[l] & hm4 ? 16 : 0));
```

```
    smin += y1[l] * sc2[0] + y1[l+32] * sc2[1] + y2[l] * sc4[0] + y2[l+32] * sc4[1];
```

```
}
```

```
    sumf += dall * (s[0] * sc1[0] + s[1] * sc1[1] + s[2] * sc3[0] + s[3] * sc3[1]) - dmin * smin;
```

```
}
```

```
sum[ith] = sumf;
```

```
//
```

```
// Accumulate the sum from all threads in the threadgroup
```

```
//
```

```

threadgroup_barrier(mem_flags::mem_threadgroup);
if (ith%4 == 0) {
    sum[ith] += sum[ith+1] + sum[ith+2] + sum[ith+3];
}
threadgroup_barrier(mem_flags::mem_threadgroup);
if (ith%16 == 0) {
    sum[ith] += sum[ith+4] + sum[ith+8] + sum[ith+12];
}
threadgroup_barrier(mem_flags::mem_threadgroup);
if (ith == 0) {
    for (int i = 16; i < nth; i += 16) sum[0] += sum[i];
    dst[r1*ne0 + r0] = sum[0];
}
}

```

Here are some differences between the `kernel_mul_mat_q3_k_f32` and `kernel_mul_mat_q5_k_f32` kernels and some suggestions that may help you debug the issues in `kernel_mul_mat_q3_k_f32`:

- 1. Structures:** The structures `block_q3_k` and `block_q5_k` have different layouts. For `block_q3_k`, the scale values are quantized with 6 bits, while for `block_q5_k`, it appears the scales and mins are quantized with 6 bits.
- 2. Quantization Masking:** In `kernel_mul_mat_q3_k_f32`, you are creating a mask and applying bitwise shifts and logical operations to extract relevant bits for quantization. In `kernel_mul_mat_q5_k_f32`, you are applying a different set of masks and shifts. Make sure that these masks and shifts are in line with how the data was originally quantized.
- 3. Data Access Pattern:** `kernel_mul_mat_q3_k_f32` accesses `x[i].scales` as `uint32_t`, while `kernel_mul_mat_q5_k_f32` accesses `(x + i)->scales` as `uint16_t`. Be sure that your data is aligned appropriately, and you are using the correct types for reading the values.
- 4. Summing:** The way summation is done over the scales seems different. In `kernel_mul_mat_q3_k_f32`, you are summing 4 elements and multiplying by `dall`, while in `kernel_mul_mat_q5_k_f32`, the summation seems to be more complicated, and there is also an additional variable `dmin`. Make sure that the summation and multiplication with scales are done in a way that is consistent with how the quantization was designed.

5. **Barriers and Accumulation:** Both kernels are using threadgroup barriers and accumulating sums within threadgroups in a similar way, so this part seems mostly equivalent.
6. **Constants:** The constants such as masks and shifts are different in both kernels which is expected if they are handling the data differently. However, you need to ensure that the constants used in `kernel_mul_mat_q3_k_f32` are correct for your data and the way it was quantized.
7. **Debugging Suggestion:** Debugging GPU kernels can be challenging. A good way to isolate issues is to progressively simplify the kernel until it becomes a minimal example that still reproduces the problem. You can start by setting static values instead of computing them, simplifying loops, etc.
8. **Attention to Detail:** Make sure that the variables in the loop of `kernel_mul_mat_q3_k_f32` that are being used to index or calculate addresses (such as `q`, `h`, `y`, etc.) are actually using the intended values. Any small typo or error can lead to unintended memory access patterns which might not necessarily cause a crash, but will cause incorrect behavior.
9. **Check the Input Data:** Sometimes, the issue is not in the kernel code but in the way the data is fed into the kernel. Make sure that your input data is correctly formatted and aligned, and that any buffers you are using are correctly set up.